

TP4 CARACTÉRISATION ET DIMENSIONNEMENT DES CACHES

C. Système mémoire presque parfait

Question C1 : Lancez la simulation, avec des caches très grands : 256 sets, 16 mots par ligne, et 4 niveaux d'associativité soit une capacité de 64 Koctets pour chacun des deux caches. On choisit également un tampon d'écritures postées de profondeur 8 mots de 32 bits. Quel est le temps d'exécution de l'application?

Le temps d'exécution est de 74502 cycles.

Question C2 : Relancez la simulation, et déterminez les taux de MISS pour le cache d'instruction et le cache de données ainsi que la valeur du CPI. Les taux de MISS sont-ils constants au cours du temps ? Comment évoluent-ils au cours des 1000 premiers cycles? interprétez ce comportement.

Taux de MISS cache instruction : 0.000654265

Taux de MISS cache de données : 0.00138131

CPI : 1.31914

Vérifier si les taux de MISS sont constants au cours du temps :

Pour vérifier sur les taux de MISS sont constants au cours de la simulation, on va échantillonner les informations à un intervalle de 10 000 cycles :

CYCLE	CPI	TAUX DE MISS INSTRUCTIONS	TAUX DE MISS DONNEES
10218	1.46034	0.00485	0.0061
20030	1.37362	0.0023	0.0029
47101		0.00095	0.00175

Pour une analyse plus précise des 1000 premiers cycles :

- Cycle 100 : CPI = 4.76, IMISS RATE = 0.1 , DMISS RATE = 0.129
- Cycle 200 : CPI = 4.28, IMISS RATE = 0.09 , DMISS RATE = 0.095
- Cycle 500 : CPI = 3.43, IMISS RATE = 0.008 , DMISS RATE = 0.066
- Cycle 750 : CPI = 2.76, IMISS RATE = 0.045 , DMISS RATE = 0.048
- Cycle 1000 : CPI = 2.42, IMISS RATE = 0.034 , DMISS RATE = 0.036

Le taux de MISS décroît significativement durant les 1000 premiers cycles.

Il n'y a qu'une seule MIPS ou processeur durant la simulation.

On peut y mettre l'hypothèse que, à l'initialisation, les instructions et les données ne sont pas chargées dans le cache.

Question C3 : En analysant le contenu des fichiers `pibus_mips32_xcache.cpp` et `pibus_mips32_xcache.h`, expliquez comment sont calculés le CPI et les deux taux de MISS.

Une fonction 'transition' sur l'instance `pibus_mips32_xcache` est appelée et met à jour des attributs d'instrumentation pour mesurer le CPI et les taux de miss. Le CPI est mesuré en divisant le compteur de cycles par le compteur d'instructions. Les taux de MISS ont pour équation : le compteur d'instructions divisé par le compteur de miss.

D. Influence de la capacité du cache instruction

Question D1 : Relevez, pour chaque configuration, la durée d'exécution du programme, le taux de MISS sur le cache d'instruction, le coût du miss instruction et la valeur du CPI. Comment interprétez ces résultats?

- **ISETS : 256 :**
cycle = 85956 , CPI = 1.54, taux de MISS = 1.48% , cout du miss = 15.31
- **ISETS : 64 :**
cycle = 106270 , CPI = 1.95, taux de MISS = 3.88% cout du miss = 15.70
- **ISETS : 16 :**
cycle = 152943 , CPI = 2.98, taux de MISS = 10.00% cout du miss = 15.61
- **ISETS : 4 :**
cycle = 166455 , CPI = 3.29, taux de MISS = 12.05%, cout du miss = 15.5
- **ISETS : 1 :**
cycle = 234079 , CPI = 5.00, taux de MISS = 15.09% cout du miss = 15.25

L'objectif est d'obtenir la plus faible valeur pour les variables.

Par les résultats, on remarque que le CPI dépend du taux de MISS. L'augmentation du taux de MISS fait augmenter le traitement d'une instruction. De plus, l'augmentation du nombre de lignes dans le cache réduit le taux de MISS et la valeur du CPI.

De ce fait, celui qui respecte le mieux les critères est **ISETS = 256**.

On peut interpréter ces résultats par le fait que le cache a une plus grande place, il peut stocker plus de données permettant d'éviter de faire de requête à la RAM et de faire des transfert de données.

Question D2 : Pourquoi la valeur obtenue pour le coût du MISS est-elle différente de la valeur estimée de la valeur estimée dans le TP3 ? Comment expliquez-vous que le coût du miss puisse avoir une valeur non entière ? Pourquoi tous les MISS n'ont-ils pas le même coût?

Dans le TP3, nous avons vu deux situations, la première est Taux de MISS presque nulle car les données se mettaient sur des lignes différentes, et la deuxième est taux de MISS 100% car les données étaient en concurrence sur la même ligne de données.

Expliquer pourquoi on ne peut avoir un taux de MISS presque nulle,

Expliquer pourquoi on ne peut pas avoir taux de MISS 100% :

prenons ISETS = 1 qui possède le pire efficacité, une ligne possède 8 cases mots.

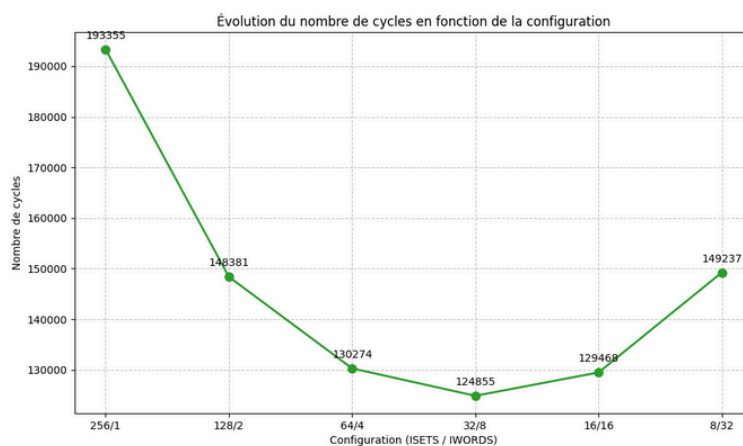
Le MIPS peut lire 8 données valide avant d'obtenir un MISS.

$100/8 = 12.5\%$ de chance de MISS

E. Influence de la largeur de la ligne de cache

Question E1 : Représenter graphiquement la durée d'exécution du programme en fonction de la largeur de la ligne de cache. Quelle est la configuration la plus efficace ? Comment expliquez-vous ce résultat ?

ISETS	IWORDS	CYCLE	CPI	TAUX DE MISS	COÛT DU MISS
256	1	193355	4.91	18.01%	7.20
128	2	148381	3.12	19.08%	8.62
64	4	130274	2.53	9.81%	11.47
32	8	124855	2.34	6.18%	15.78
16	16	129468	2.38	4.28%	24.22
8	32	149237	2.73	3.47%	39.91



Par analyse du graphique, nous remarquons que la meilleure configuration est ISETS = 32 et IWORDS = 8 qui optimise à la fois la localité spatiale et la localité temporelle.

Le cas de ISETS = 256 et IWORDS = 1, il maximise le nombre de ligne du cache implique de devoir faire une requête à la RAM pour chaque donnée non connu dans le cache. $\Leftrightarrow$ Il n'utilise pas la localité spatiale.

Le cas de ISETS = 8 et IWORDS = 32, il maximise la localité spatiale mais durant la simulation, le programme a besoin de garder les anciennes données.

F. Influence de la capacité du cache de données

Question F1 : Relevez, pour chaque valeur, la durée d'exécution du programme, le taux de MISS sur le cache de données, le coût du miss de données et la valeur du CPI.

ISETS	IWORDS	DSETS	CYCLE	CPI	TAUX DE MISS	COÛT DU MISS
256	16	256	74553	1.31	0.25%	17.76
256	16	64	74638	1.32	0.25%	17.82
256	16	16	94444	1.67	7.15%	18.40
256	16	4	144880	2.56	22%	16.67
256	16	1	218474	3.86	38.04%	15.85

G. Influence de la profondeur du tampon d'écritures postées

Question G1 : Comment est réalisé le tampon d'écriture postées? Quelles sont les informations qu'il faut stocker dans le tampon pour chaque requête enregistrée? Que se passe-t-il lorsque le processeur effectue une requête d'écriture alors que le tampon d'écriture postées est plein? Que se passe-t-il lorsque le processeur fait une requête de lecture (instruction ou donnée) qui fait miss, alors que le tampon d'écriture est non-vidé ? Pourquoi ce comportement ?

- **Réalisation du tampon :** Le tampon d'écritures postées est un mécanisme de file d'attente de type FIFO (First In, First Out).
- **Informations à stocker :** Pour chaque requête d'écriture, le tampon doit enregistrer l'adresse mémoire de destination, la donnée à écrire, la taille du transfert et des bits de contrôle
- **Comportement si le tampon est plein :** Si le processeur tente une écriture alors que le tampon a atteint sa capacité maximale (WBUF_DEPTH), le processeur est mis en attente (gel ou *stall*). Il doit patienter jusqu'à ce que le contrôleur mémoire ait terminé d'écrire au moins une donnée du tampon vers la mémoire principale pour libérer une place.
- **Comportement lors d'un MISS en lecture avec un tampon non-vidé :** priorité d'abord au requête d'écriture avant de charger les données. Le contrôleur de cache doit impérativement vider l'intégralité du tampon d'écritures vers la mémoire principale *avant* d'aller chercher la nouvelle ligne de cache manquante.
- **Raison de ce comportement :** C'est une question de cohérence mémoire (aléa Read-After-Write). Si le processeur vient d'écrire une donnée à l'adresse X (actuellement stockée dans le tampon d'écriture) et qu'il fait ensuite une lecture qui provoque un MISS nécessitant de lire l'adresse X en mémoire principale, il risquerait de lire l'ancienne valeur si le tampon n'a pas été vidé au préalable.

Question G2 : Mesurez le temps d'exécution de l'application, le CPI, le coût des écritures et la fréquence des écritures pour des profondeurs de 1, 2, 4 et 8 mots. A quoi correspond le coût des écritures ? Comment expliquez-vous le fait que le coût des écritures soit si faible ? J'ai pris pour paramètre : NSET = 256 , NWAYS = 8, NWORDS = 8 pour chacun des deux caches.

WBUF_DEPTH	CYCLE	CPI	WRITE_COST	WRITE_RATE
8	74553	1.31	0	12.5%
4	74553	1.31	0	12.5%
2	75497	1.33	0.13	12.5%
1	77665	1.37	0.51	12.5%

Dans cette première situation, la différence n'est pas visible à cause de l'optimisation par les propriétés de localité spatial et temporel.

- **À quoi correspond le coût des écritures ?** Le coût des écritures représente le temps (en cycles) pendant lequel le processeur est bloqué (gelé) à cause d'une opération d'écriture. Avec un tampon d'écritures postées, le processeur n'attend pas que la donnée atteigne la mémoire lente. Il subit un coût uniquement dans deux cas de figure : lorsque le tampon est plein au moment de l'écriture, ou lorsqu'un MISS en lecture oblige le système à vider le tampon avant de résoudre le MISS.
- **Comment expliquer que ce coût soit si faible ?** Le but d'un tampon d'écritures postées est précisément de masquer la latence de la mémoire principale. Tant que le processeur n'écrit pas de données en rafale (à une vitesse supérieure à la bande passante de la mémoire) ou qu'il ne subit pas un MISS en lecture immédiatement après une écriture, le tampon a le temps de se vider "en arrière-plan". Du point de vue du processeur, l'écriture est quasi-instantanée puisqu'il se contente de déposer sa requête dans le tampon et passe immédiatement à l'instruction suivante. Plus le tampon est profond, plus il est capable d'absorber des rafales d'écritures, rapprochant ainsi le coût moyen de l'écriture de zéro.